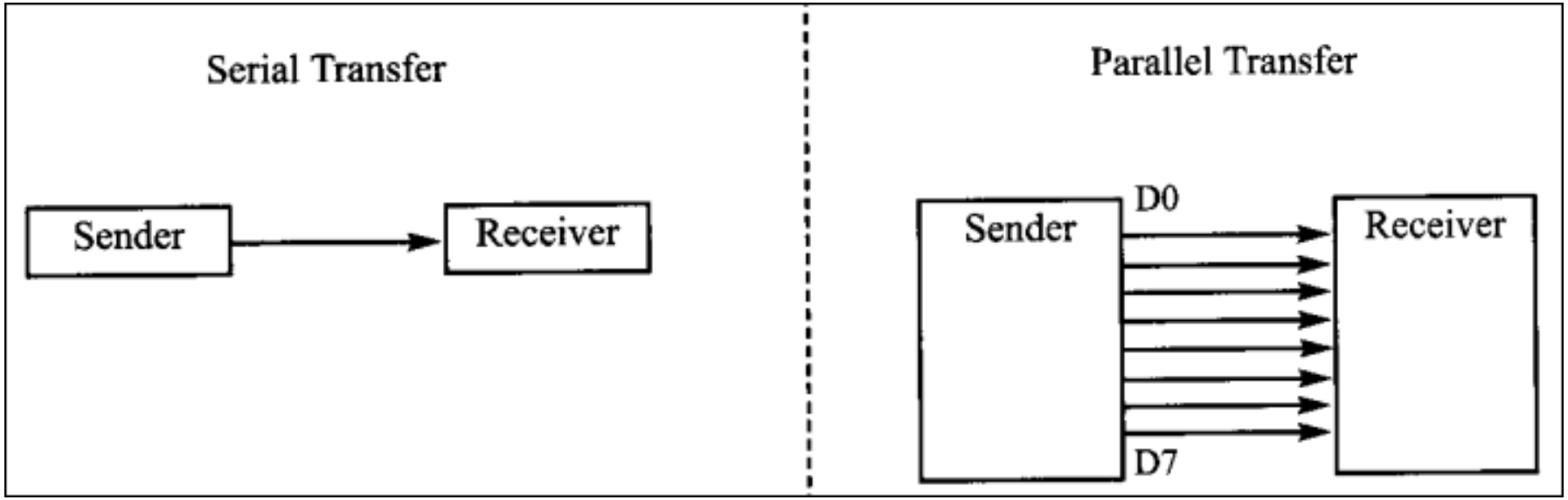
AVR MEGA 2560 Serial Port

Computer transfer data in two ways:

- Parallel
- Serial



For Serial Communication, the byte of data must be converted to serial bits using parallel-in-serial-out shift registers. Then it can be transmitted over a single data line.

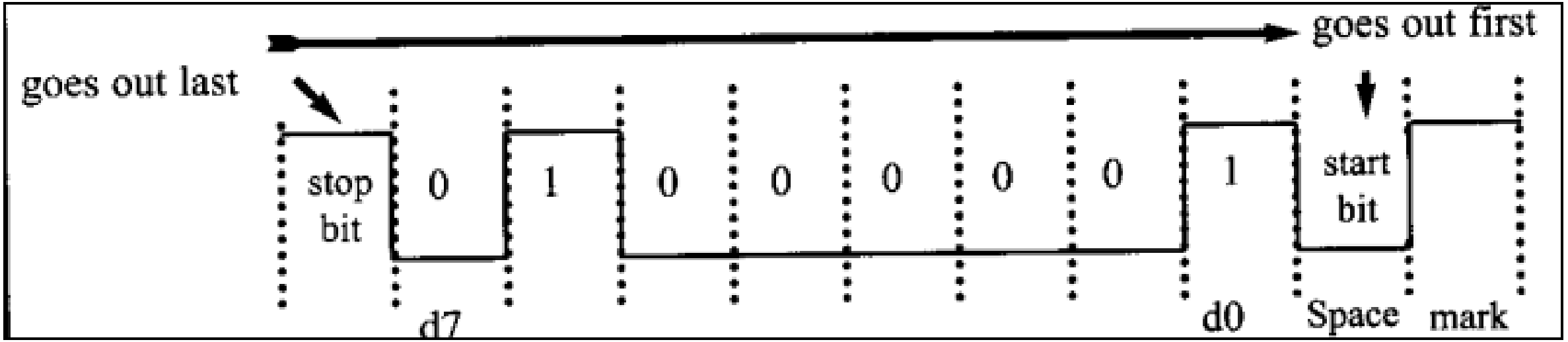
Serial data communication uses two methods:

- Asynchronous
- Synchronous

Asynchronous method transfers a single byte (character) at a time.
Synchronous method transfers a block of data byte (characters) at a time.

<u>Asynchronous serial Communication and data framing:</u>

-Start and Stop bits -Parity Bit



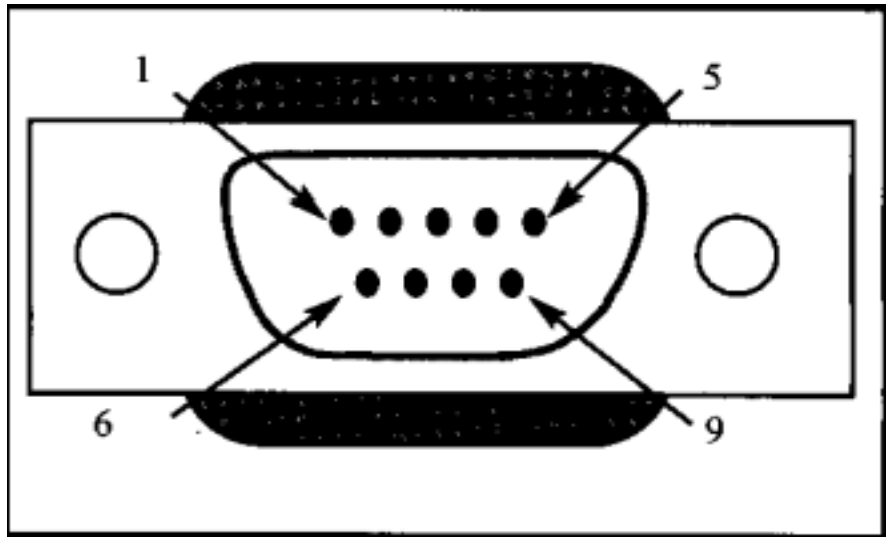
Data Transfer Rate:

The rate of data transfer in serial data communication is stated in bits per second (bps) or baud rate.

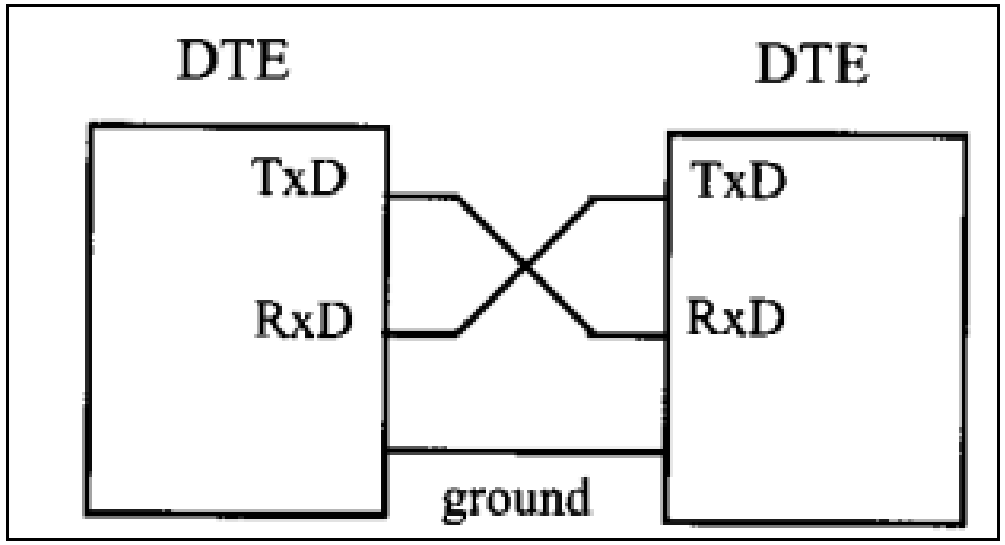
RS232 Standard:

- For compatibility across various manufacturers, an interfacing standard called RS232 was set by the Electronics Industries Association (EIA) in 1960
- Standard was set before the advent of TTL logic family
- In RS232, a 1 is -3V to -25V and a 0 is +3V to +25V
- To connect any RS232 to a microcontroller system, voltage converter (line drivers) such as MAX232 is used

RS232 Pins:



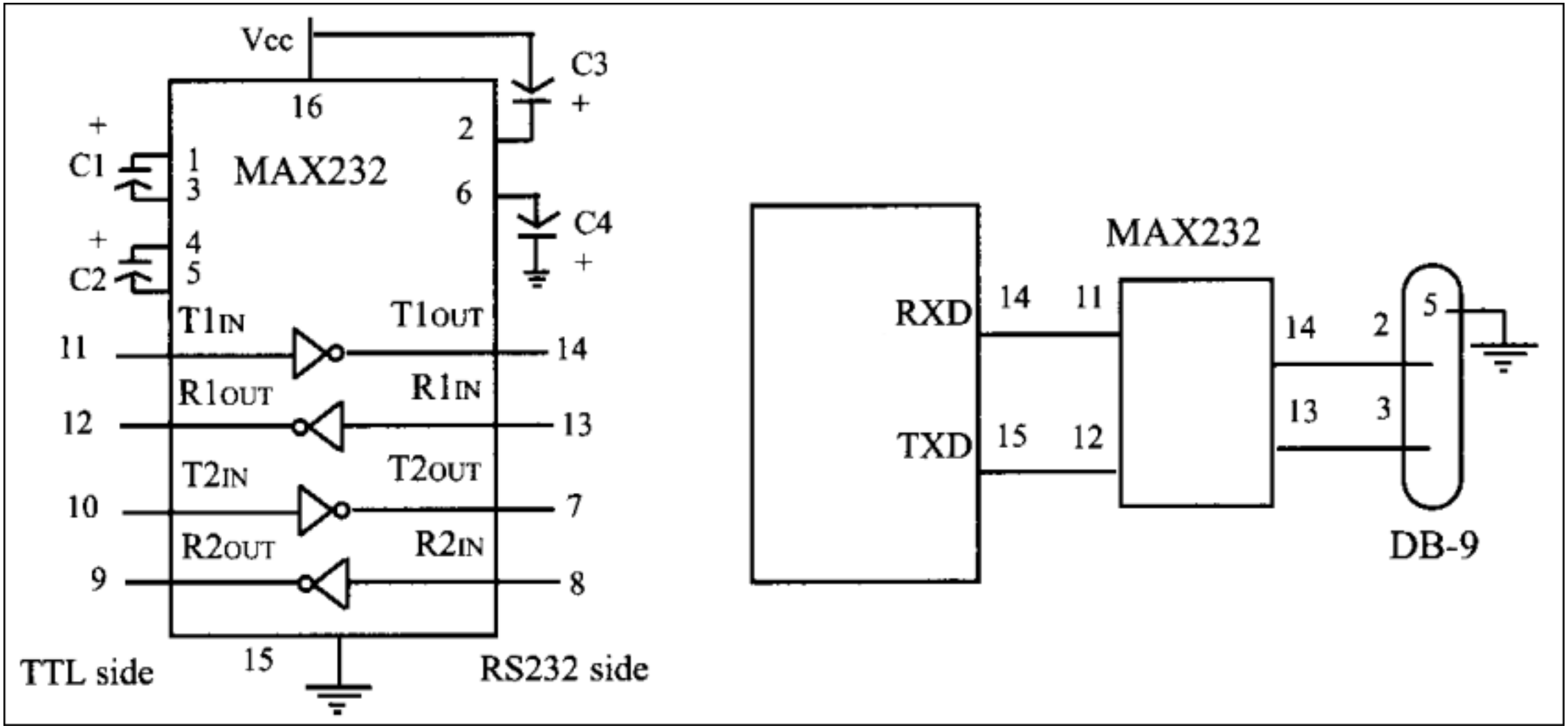
Pin	Description
1	Data carrier detect (DCD)
2	Received data (RxD)
3	Transmitted data (TxD)
4	Data terminal ready (DTR)
5	Signal ground (GND)
6	Data set ready (DSR)
7	Request to send (RTS)
8	Clear to send (CTS)
9	Ring indicator (RI)



MEGA 2560 Connections to MAX232:

MEGA 2560 has four serial communication ports for transferring and receiving data USART0, USART1, USART2 and USART3.

Each has TXn and RXn pins.



Capacitors ranges from 0.1µF to 22µF

Serial Port Programming Registers:

Six registers are associated with USART: (n = 0-3)

- UDRn
- UCSRnA
- UCSRnB
- UCSRnC
- UBRRnH
- UBRRnL

UDRn – USART I/O Data Register n :

Bit	7	6	5	4	3	2	1	0	
	RXB[7:0]								UDRn (Read)
	TXB[7:0]								UDRn (Write)
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The USART Transmit Data Buffer Register and USART Receive Data Buffer Registers share the same I/O address referred to as USART Data Register or UDRn. The Transmit Data Buffer Register (TXB) will be the destination for data written to the UDRn Register location. Reading the UDRn Register location will return the contents of the Receive Data Buffer Register (RXB).
For 5-bit, 6-bit, or 7-bit characters the upper unused bits will be ignored by the Transmitter and set to zero by the Receiver.

UCSRnA – USART Control and Status Register A:

Bit	7	6	5	4	3	2	1	0	
	RXCn	TXCn	UDREn	FEn	DORn	UPEn	U2Xn	MPCMn	UCSRnA
Read/Write	R	R/W	R	R	R	R	R/W	R/W	
Initial Value	0	0	1	0	0	0	0	0	

- **Bit 7 – RXCn: USART Receive Complete**

This flag bit is set when there are unread data in the receive buffer and cleared when the receive buffer is empty (that is, does not contain any unread data). If the Receiver is disabled, the receive buffer will be flushed and consequently the RXCn bit will become zero. The RXCn Flag can be used to generate a Receive Complete interrupt.

- **Bit 6 – TXCn: USART Transmit Complete**

This flag bit is set when the entire frame in the Transmit Shift Register has been shifted out and there are no new data currently present in the transmit buffer (UDRn). The TXCn Flag bit is automatically cleared when a transmit complete interrupt is executed, or it can be cleared by writing a one to its bit location. The TXCn Flag can generate a Transmit Complete interrupt.

- **Bit 5 – UDREn: USART Data Register Empty**

The UDREn Flag indicates if the transmit buffer (UDRn) is ready to receive new data. If UDREn is one, the buffer is empty, and therefore ready to be written. The UDREn Flag can generate a Data Register Empty interrupt.
UDREn is set after a reset to indicate that the Transmitter is ready.

- **Bit 4 – FEn: Frame Error**

This bit is set if the next character in the receive buffer had a Frame Error when received, that is, when the first stop bit of the next character in the receive buffer is zero. This bit is valid until the receive buffer (UDRn) is read. The FEn bit is zero when the stop bit of received data is one. Always set this bit to zero when writing to UCSRnA.

- **Bit 3 – DORn: Data OverRun**

This bit is set if a Data OverRun condition is detected. A Data OverRun occurs when the receive buffer is full (two characters), it is a new character waiting in the Receive Shift Register, and a new start bit is detected. This bit is valid until the receive buffer (UDRn) is read. Always set this bit to zero when writing to UCSRnA.

- **Bit 2 – UPEn: USART Parity Error**

This bit is set if the next character in the receive buffer had a Parity Error when received and the Parity Checking was enabled at that point (UPMn1 = 1). This bit is valid until the receive buffer (UDRn) is read. Always set this bit to zero when writing to UCSRnA.

- **Bit 1 – U2Xn: Double the USART Transmission Speed**

This bit only has effect for the asynchronous operation. Write this bit to zero when using synchronous operation. Writing this bit to one will reduce the divisor of the baud rate divider from 16 to 8 effectively doubling the transfer rate for asynchronous communication.

- **Bit 0 – MPCMn: Multi-processor Communication Mode**

This bit enables the Multi-processor Communication mode. When the MPCMn bit is written to one, all the incoming frames received by the USART Receiver that do not contain address information will be ignored. The Transmitter is unaffected by the MPCMn setting.

UCSRnB – USART Control and Status Register n B:

Bit	7	6	5	4	3	2	1	0	
	~~RXCIEn~~	~~TXCIEn~~	~~UDRIEn~~	RXENn	TXENn	~~UCSZn2~~	~~RXB8n~~	~~TXB8n~~	UCSRnB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – RXCIEn: RX Complete Interrupt Enable n**

Writing this bit to one enables interrupt on the RXCn Flag. A USART Receive Complete interrupt will be generated only if the RXCIEn bit is written to one, the Global Interrupt Flag in SREG is written to one and the RXCn bit in UCSRnA is set.

- **Bit 6 – TXCIEn: TX Complete Interrupt Enable n**

Writing this bit to one enables interrupt on the TXCn Flag. A USART Transmit Complete interrupt will be generated only if the TXCIEn bit is written to one, the Global Interrupt Flag in SREG is written to one and the TXCn bit in UCSRnA is set.

- **Bit 5 – UDRIEn: USART Data Register Empty Interrupt Enable n**

Writing this bit to one enables interrupt on the UDREn Flag. A Data Register Empty interrupt will be generated only if the UDRIEn bit is written to one, the Global Interrupt Flag in SREG is written to one and the UDREn bit in UCSRnA is set.

- **Bit 4 – RXENn: Receiver Enable n**

Writing this bit to one enables the USART Receiver. The Receiver will override normal port operation for the RxDn pin when enabled. Disabling the Receiver will flush the receive buffer invalidating the FEn, DORn, and UPEn Flags.

- **Bit 3 – TXENn: Transmitter Enable n**

Writing this bit to one enables the USART Transmitter. The Transmitter will override normal port operation for the TxDn pin when enabled. The disabling of the Transmitter (writing TXENn to zero) will not become effective until ongoing and pending transmissions are completed, that is, when the Transmit Shift Register and Transmit Buffer Register do not contain data to be transmitted. When disabled, the Transmitter will no longer override the TxDn port.

- **Bit 2 – UCSZn2: Character Size n**

The UCSZn2 bits combined with the UCSZn1:0 bit in UCSRnC sets the number of data bits (Character SiZe) in a frame the Receiver and Transmitter use.

- **Bit 1 – RXB8n: Receive Data Bit 8 n**

RXB8n is the ninth data bit of the received character when operating with serial frames with nine data bits. Must be read before reading the low bits from UDRn.

- **Bit 0 – TXB8n: Transmit Data Bit 8 n**

TXB8n is the ninth data bit in the character to be transmitted when operating with serial frames with nine data bits. Must be written before writing the low bits to UDRn.

UCSRnC – USART Control and Status Register n C:

Bit	7	6	5	4	3	2	1	0	
	UMSELn1	UMSELn0	UPMn1	UPMn0	USBSn	UCSZn1	UCSZn0	[redacted]	UCSRnC
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	1	1	0	

- **Bits 7:6 – UMSELn1:0 USART Mode Select**

These bits select the mode of operation of the USARTn

UMSELn1	UMSELn0	Mode
0	0	Asynchronous USART
0	1	Synchronous USART
1	0	(Reserved)
1	1	Master SPI (MSPIM)(1)

- **Bits 5:4 – UPMn1:0: Parity Mode**

These bits enable and set type of parity generation and check. If enabled, the Transmitter will automatically generate and send the parity of the transmitted data bits within each frame. The Receiver will generate a parity value for the incoming data and compare it to the UPMn setting. If a mismatch is detected, the UPEn Flag in UCSRnA will be set.

UPMn1	UPMn0	Parity Mode
0	0	Disabled
0	1	Reserved
1	0	Enabled, Even Parity
1	1	Enabled, Odd Parity

- **Bit 3 – USBSn: Stop Bit Select**

This bit selects the number of stop bits to be inserted by the Transmitter. The Receiver ignores this setting.

USBSn	Stop Bit(s)
0	1-bit
1	2-bit

- **Bit 2:1 – UCSZn1:0: Character Size**

The UCSZn1:0 bits combined with the UCSZn2 bit in UCSRnB sets the number of data bits (Character SiZe) in a frame the Receiver and Transmitter use.

UCSZn2	UCSZn1	UCSZn0	Character Size
0	0	0	5-bit
0	0	1	6-bit
0	1	0	7-bit
0	1	1	8-bit
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Reserved
1	1	1	9-bit

- **Bit 0 – UCPOLn: Clock Polarity**

This bit is used for synchronous mode only. Write this bit to zero when asynchronous mode is used. The UCPOLn bit sets the relationship between data output change and data input sample, and the synchronous clock (XCKn).

UBRRnL and UBRRnH – USART Baud Rate Registers:

Bit	15	14	13	12	11	10	9	8	
	–	–	–	–	UBRR[11:8]				UBRRHn
	UBRR[7:0]								UBRRLn
	7	6	5	4	3	2	1	0	
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
	R/W	R/W	R/W	R/W	R/W	R/W			
Initial Value	0	0	0	0	0	0			
	0	0	0	0	0	0			

- **Bit 15:12 – Reserved Bits**

These bits are reserved for future use. For compatibility with future devices, these bit must be written to zero when UBRRH is written.

- **Bit 11:0 – UBRR11:0: USART Baud Rate Register**

This is a 12-bit register which contains the USART baud rate. The UBRRH contains the four most significant bits, and the UBRRL contains the eight least significant bits of the USART baud rate. Ongoing transmissions by the Transmitter and Receiver will be corrupted if the baud rate is changed. Writing UBRRL will trigger an immediate update of the baud rate prescaler.

Baud Rate [bps]	f_{osc} = 16.0000MHz			
	U2Xn = 0		U2Xn = 1	
	UBRR	Error	UBRR	Error
2400	416	-0.1%	832	0.0%
4800	207	0.2%	416	-0.1%
9600	103	0.2%	207	0.2%
14.4K	68	0.6%	138	-0.1%
19.2K	51	0.2%	103	0.2%
28.8K	34	-0.8%	68	0.6%
38.4K	25	0.2%	51	0.2%
57.6K	16	2.1%	34	-0.8%
76.8K	12	0.2%	25	0.2%
115.2K	8	-3.5%	16	2.1%
230.4K	3	8.5%	8	-3.5%
250K	3	0.0%	7	0.0%
0.5M	1	0.0%	3	0.0%
1M	0	0.0%	1	0.0%
Max.(1)	1Mbps		2Mbps	

Programming the AVR to Transfer data serially:

In programming the AVR to transfer character bytes serially, the following steps must be taken:

1. The UCSRnB register is loaded with the value 08H, enabling the USART transmitter. The transmitter will override normal port operation for the TxDn pin when enabled
2. The UCSRnC register is loaded with the value 06H, indicating asynchronous mode with 8-bit data frame, no parity and one stop bit
3. The UBRRn is loaded with one of the values in Table (if Fosc = 16MHz) to set the baud rate for serial data transfer
4. The character byte to be transmitted serially is written into the UDRn register
5. Monitor the UDRE bit of UCSRnA register to make sure UDRn is ready for the next byte
6. To transmit the next character, go to step 4

Write a C program for the AVR to transfer the letter ‘G’ serially at 9600 baud, continuously. Use 8-bit data, and 1 stop bit. Assume Crystal Frequency is 16MHz

```
#include<avr/io.h>
void usart_init (void)
{
        UBRR1H = 0x00;
        UBRR1L = 0x67;
        UCSR1B = (1<<TXEN1); 0x08/for transmitter enable
        UCSR1C = (1<<UCSZ11) | (1<<UCSZ10); USCR1c=0x06; 8 bit character size selection
}
void usart_send(unsigned char ch)
{
        while(! (UCSR1A & (1<<UDRE1)));
        UDR1 = ch;
}
int main (void)
{
        usart_init();
        while(1)
        {
                usart_send('G');
        }
        return 0;
}
```

Write a program to send the message “The Earth is but one country” serially at 9600 baud, continuously. Use 8-bit data, and 1 stop bit. Assume Crystal Frequency is 16MHz

```
#include<avr/io.h>
void usart_init (void)
{
        UBRR1H = 0x00;
        UBRR1L = 0x67;
        UCSR1B = (1<<TXEN1);
        UCSR1C = (1<<UCSZ11) | (1<<UCSZ10);
}
void usart_send(unsigned char ch)
{
        while(! (UCSR1A & (1<<UDRE1)));
        UDR1 = ch;
}
int main (void)
{
        unsigned char str[30] = "The Earth is but one country";
        unsigned char strlength = 30;
        unsigned char i = 0;
        usart_init();
        while(1)
        {
                usart_send(str[i++]);
                if (i >= strlength)
                i = 0;
        }
        return 0;
}
```

Programming the AVR to Receive data serially:

In programming the AVR to receive character bytes serially, the following steps must be taken:

1. The UCSRnB register is loaded with the value 10H, enabling the USART receiver. The receiver will override normal port operation for the RxDn pin when enabled
2. The UCSRnC register is loaded with the value 06H, indicating asynchronous mode with 8-bit data frame, no parity and one stop bit
3. The UBRRn is loaded with one of the values in Table (if Fosc = 16MHz) to set the baud rate for serial data transfer
4. The RXC flag bit of the UCSRnA register is monitored for a HIGH to see if any character has been received yet
5. When RXC is raised, the UDRn register has the byte. Its contents are moved into a safe place
6. To receive the next character, go to step 5

Write a C program for the AVR to receive bytes of data serially and send them on PORTK. Set the baud rate at 9600 baud, 8-bit data, and 1 stop bit. Assume Crystal Frequency is 16MHz

```
#include<avr/io.h>
int main (void)
{
        DDRK = 0xFF;
        UBRR1L = 0x67;
        UCSR1B = (1<<RXEN1);
        UCSR1C = (1<<UCSZ11) | (1<<UCSZ10);
        while(1)
        {
                while(! (UCSR1A & (1<<RXC1)));
                PORTK = UDR1;
        }
        return 0;
}
```

Write a C program for the AVR to receive characters from the serial port. If it is ‘a’ – ‘z’ change it to a capital letters and transmit it back. Set the baud rate at 9600 baud, 8-bit data, and 1 stop bit.

Do By Yourself

Transmit and Receive

```
#include<avr/io.h>
void usart_send(unsigned char ch1)
{
		while(! (UCSR1A & (1<<UDRE1)));
		UDR1 = ch1;
}
void usart_init(void)
{
	UBRR1H = 0x00;
	UBRR1L = 0x67;
		UCSR1B = (1<<TXEN1)| (1<<RXEN1);
		UCSR1C = (1<<UCSZ11) | (1<<UCSZ10);
}
int main (void)
{
		DDRK = 0xFF;
		DDRF = 0x00;
	usart_init();
		while(1)
		{
			usart_send('G');
			while(! (UCSR1A & (1<<RXC1)));
			PORTK = UDR1;
		}
		return 0;
}
```

Write a C program for the AVR to receive bytes of data serially and put them on PORTK. Set the baud rate at 9600 baud, 8-bit data, and 1 stop bit. Assume Crystal Frequency is 16MHz

```
#include<avr/io.h>
void usart_send(unsigned char ch)
{
        while(! (UCSR1A & (1<<UDRE1)));
        UDR1 = ch;
}
int main (void)
{
        DDRK = 0xFF;
        DDRF = 0x00;
        UBRR1L = 0x67;
        UCSR1B = (1<<RXEN1);
        UCSR1C = (1<<UCSZ11) | (1<<UCSZ10);
        while(1)
        {
                while(! (UCSR1A & (1<<RXC1)));
                PORTK = UDR1;
                usart_send('G');
        }
        return 0;
}
```

ON (‘a’) and OFF (‘b’) PORTF pin 5

```
#include<avr/io.h>
int main (void)
{
  DDRK = 0xFF;
  DDRF = (1<<4) | (1<<5);
  UBRR3L = 0x19;
  UCSR3B = (1<<RXEN3);
  UCSR3C = (1<<UCSZ31) | (1<<UCSZ30);
  unsigned char x;
  while(1)
  {
    while(! (UCSR3A & (1<<RXC3)));
    x = UDR3;
    if (x == 'b')
      PORTF &= ~(1<<5);
    if (x == 'a')
      PORTF |= (1<<5);

  }
  return 0;
}
```

```
#include<avr/io.h>
#include<util/delay.h>
unsigned char received;
void lcdcommand(unsigned char cmnd)
{
  PORTF &= ~(1<<0); //RS = 0 for command
  PORTK = cmnd;         //send cmnd to data port
  PORTF |= (1<<1);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  PORTF &= ~(1<<1); //EN = 1 for H-to-L pulse
  _delay_us(100);          //wait tomakeenable wide
}
//*****************************************************
void lcddata(unsigned char data)
{
  PORTF |= (1<<0);  //RS = 1 for data
  PORTK = data;     //send data to data port
  PORTF |= (1<<1);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  PORTF &= ~(1<<1); //EN = 1 for H-to-L pulse
  _delay_us(100);          //wait tomakeenable wide
}
//*****************************************************
void lcd_init()
{
```